

COUNCILia

Stack Tecnológico y Arquitectura

Tecnologías, orquestador propio y modelo de datos.

- Criterios de selección
- Stack completo
- Por qué orquestador propio
- Componentes del orquestador custom
- Modelo de datos
- Decisiones que se posponen
- Riesgos técnicos conocidos

Tabla de contenidos

1 Criterios de selección

2 Tabla de stack completo

3 Por qué orquestador propio

4 Componentes del orquestador custom

5 Modelo de datos

6 Decisiones que se posponen

7 Riesgos técnicos conocidos

Qué tecnologías se usan en el MVP, cómo está organizado el orquestador propio y cuál es el modelo de datos. Documento 6 de 9 de la serie del MVP v1.1.

1. Criterios de selección

El stack se eligió bajo tres criterios, en este orden:

1. **Velocidad para validar en 8 semanas** con un equipo full-stack pequeño.
2. **Control total sobre la capa de orquestación** (sin frameworks multiagente genéricos).
3. **Caminos claros de escala post-MVP** sin reescribir el producto.

Cualquier alternativa "moderna pero exótica" pierde frente a estos tres criterios. El stack es deliberadamente aburrido en sus piezas, e interesante solo donde importa: el orquestador.

2. Tabla de stack completo

Capa	Tecnología	Justificación
Frontend	Next.js 14 (App Router)	SSR para SEO, routing limpio, escala bien hacia móvil. TypeScript por defecto.
Estilos	Tailwind CSS	Velocidad de prototipado sin perder control de diseño. Compatible con sistema de diseño a futuro.
Animaciones	Framer Motion	Micro-interacciones de la experiencia conversacional. Streaming visual de respuestas.
Backend / API	Next.js API Routes	Mismo repositorio en MVP. Migrable a FastAPI cuando se necesite control adicional.
Orquestación	Custom Orchestrator (TS)	No usar LangChain ni CrewAI en MVP. Control total sobre la dinámica deliberativa.
LLM	Claude Sonnet (Anthropic API)	Maneja system prompts complejos, respeta instrucciones de rol, costo razonable para múltiples llamadas.
Base de datos	Supabase (PostgreSQL)	Auth integrada, storage para memoria a futuro, real-time disponible para streaming.
Autenticación	Supabase Auth	Integrada con la base de datos. Sin dependencias extra.
Hosting	Vercel	Integración nativa con Next.js, deploys automáticos, edge functions para latencia baja.
Móvil (futuro)	React Native / Expo	Reutiliza componentes y la lógica de orquestación del web.

3. Por qué orquestador propio

La decisión más importante del stack no es qué frameworks usar, sino qué framework **no** usar.

LangChain y CrewAI son los dos candidatos obvios para sistemas multiagente. Ambos quedan **fuera del MVP** por tres razones:

- **Asumen agentes colaborativos.** Su modelo mental es "varios agentes que se ayudan a completar una tarea". El nuestro es el opuesto: agentes que se contradicen para iluminar tradeoffs. El framework lucha contra el producto.
- **Añaden complejidad sin aportar control.** Los frameworks genéricos abstraen la cosa que más necesitamos iterar: la dinámica deliberativa. Cada vuelta de calibración pasaría por adaptar la API del framework en vez de la lógica del producto.
- **Versionado externo.** Iterar la deliberación dependería de versiones que no controlamos. Para un MVP en 8 semanas eso es inaceptable.

El orquestador propio escrito en TypeScript suma ~600–900 líneas en el MVP. Es claramente más barato que entender e instrumentar un framework de propósito general.

4. Componentes del orquestador custom

Componente	Responsabilidad
Intent Calibrator	Lee <code>userContext</code> + mensaje del usuario y decide pesos relativos y atenuaciones de los agentes.
Agent Runner	Ejecuta las 3 llamadas a Claude en paralelo con system prompts distintos y streaming server-sent.
Tension Detector	Identifica el par con mayor contradicción semántica entre las posturas iniciales para disparar la réplica.
Debate Router	Selecciona quién responde a quién y arma el contexto reducido (solo la postura del agente referido) para esa réplica.
Synthesis Generator	Genera la síntesis con tradeoffs explícitos cuando el usuario la pide, usando la estructura fija de salida.

Notas de implementación:

- Cada componente es una función pura cuyas dependencias se inyectan. Eso permite testearlo con fixtures sin pegarle al modelo.
- El Agent Runner usa `Promise.all` sobre 3 streams server-sent events. La cancelación coordinada es trivial porque el MVP **no implementa interrupción durante streaming** (v1.2).
- El Tension Detector del MVP es heurístico (overlap léxico negativo + clasificador ligero opcional). Reemplazable por un LLM-as-judge en v1.2 si la calidad lo justifica.

5. Modelo de datos

Cuatro entidades principales en Supabase / PostgreSQL:

Entidad	Campos principales
<code>users</code>	<code>id</code> · <code>email</code> · <code>created_at</code> · <code>plan</code> · <code>onboarding_completed_at</code>
<code>councils</code>	<code>id</code> · <code>user_id</code> · <code>user_context</code> (JSON) · <code>created_at</code>
<code>conversations</code>	<code>id</code> · <code>council_id</code> · <code>user_id</code> · <code>title</code> · <code>status</code> · <code>created_at</code>
<code>messages</code>	<code>id</code> · <code>conversation_id</code> · <code>agent_id</code> · <code>role</code> · <code>phase</code> · <code>content</code> · <code>token_count</code> · <code>timestamp</code> · <code>replies_to_agent_id</code>

Notas:

- `agent_id` es un enum: `marco` | `elena` | `rafael` | `user` | `synthesis`.
- `phase` es un enum: `initial` | `reply` | `synthesis` | `user_input`.
- `user_context` se guarda **por council**, no por sesión, para que sesiones futuras del mismo council reutilicen el contexto.
- `replies_to_agent_id` es opcional y solo se llena en mensajes de la fase `reply`.

6. Decisiones que se posponen

Cosas que **no se construyen en el MVP** y deben quedar fuera del scope, aunque sean tentadoras:

- **Memoria persistente entre sesiones.** Los agentes no "recuerdan" sesiones anteriores. Se introduce en v1.3.
- **Embeddings y recuperación semántica.** El MVP no usa vectores; las sesiones son autocontenidas.
- **Caching de respuestas.** El producto vive de respuestas únicas; el cache aporta poco y suma complejidad.
- **Multi-modelo.** Solo Claude Sonnet en el MVP. Probar otro modelo es una decisión post-validación.
- **Self-hosting del LLM.** No se considera hasta tener métricas reales de costo y volumen.

7. Riesgos técnicos conocidos

Riesgos a vigilar durante el sprint del MVP:

- **Latencia de 3 llamadas paralelas.** Si Claude Sonnet tiene tiempos de cola variables, la fase 1 sentirá lento. Mitigación: instrumentar percentiles, presupuesto < 6 s para que los 3 terminen.
- **Tension Detector con falsos negativos.** Si declara "no hay contradicción" en >15% de los turnos, la réplica deja de aparecer y el producto se siente plano. Mitigación: dashboard de tasa de réplica omitida desde el día 1.
- **Costo por sesión.** Estimado realista en \$0.10–0.20 USD, 2–3× el estimado del MVP original. El modelo de negocio del freemium debe ser por sesiones, no por mensajes (ver documento 7 — Operaciones).
- **Vendor lock-in con Anthropic.** Mitigación: la capa de Agent Runner expone una interfaz [LLM](#) propia; cambiar de proveedor implica una sola implementación nueva.